Labsheet 5 : BTCS301P-AS05

Computer Vision Lab

Khushi Choudhary CU240251564

In [ ]:
```
import cv2
import numpy as np
import matplotlib.pyplot as plt
from skimage.feature import hog
from skimage import exposure, data

# Step 1: Load real-world sample images
img1 = data.astronaut()
rows, cols, _ = img1.shape
M = cv2.getRotationMatrix2D((cols/2, rows/2), 30, 0.8) # 30 deg rotation, 0.8 scale
img2 = cv2.warpAffine(img1, M, (cols, rows))

# Step 2: Convert images to grayscale
gray1 = cv2.cvtColor(img1, cv2.COLOR_RGB2GRAY)
gray2 = cv2.cvtColor(img2, cv2.COLOR_RGB2GRAY)
gray1_blur = cv2.GaussianBlur(gray1, (3, 3), 0)
gray2_blur = cv2.GaussianBlur(gray2, (3, 3), 0)
print(f"Image 1 Shape: {gray1.shape}, Image 2 Shape: {gray2.shape}")
```

```
Image 1 Shape: (512, 512), Image 2 Shape: (512, 512)
```

In [18]:
```
# Step 3: Initialize SIFT Detector
sift = cv2.SIFT_create()
kp1, des1 = sift.detectAndCompute(gray1_blur, None)
kp2, des2 = sift.detectAndCompute(gray2_blur, None)

print(f"Total SIFT Keypoints detected in Image 1: {len(kp1)}")
print(f"Descriptor Shape: {des1.shape} (128-D vector per keypoint)")

# Step 4: Visualize keypoints with scale and orientation flags
img_sift_keypoints = cv2.drawKeypoints(
    img1, kp1, None,
    flags=cv2.DRAW_MATCHES_FLAGS_DRAW_RICH_KEYPOINTS
)
plt.figure(figsize=(5, 4))
plt.imshow(cv2.cvtColor(img_sift_keypoints, cv2.COLOR_BGR2RGB))
plt.title("SIFT Keypoints (Circle size = scale, Line = orientation)")
plt.axis('off')
plt.show()
```

```
Total SIFT Keypoints detected in Image 1: 1001
Descriptor Shape: (1001, 128) (128-D vector per keypoint)
```

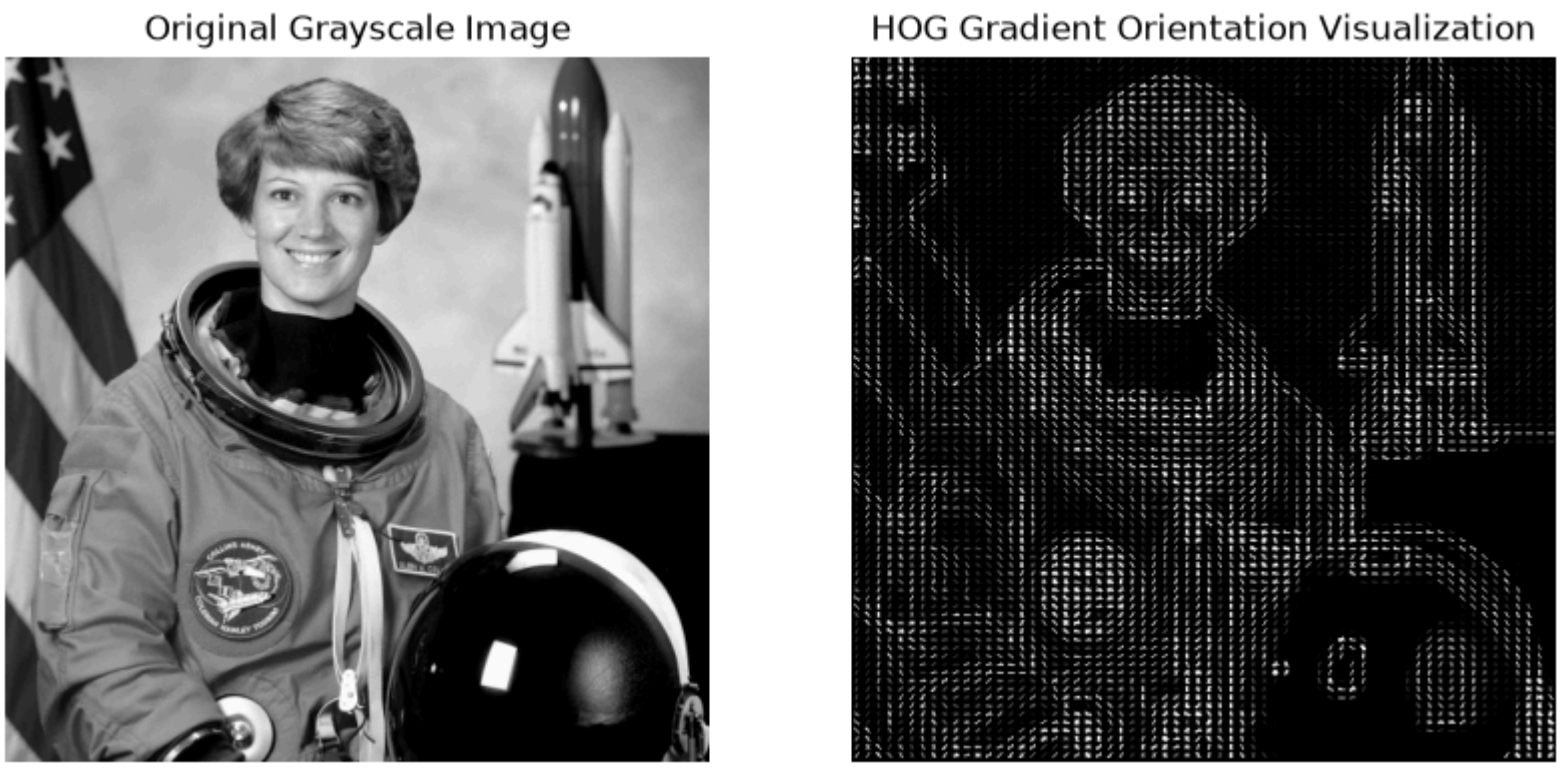


In [15]:
```
# Step 5: Extract HOG Features using scikit-image
hog_features, hog_image = hog(
    gray1_blur,
    orientations=9,               # 9 orientation bins (0 to 180 degrees)
    pixels_per_cell=(6, 8),       # 8x8 pixel cells
    cells_per_block=(2, 2),       # 2x2 cells per block for normalization
    visualize=True,               # Generate visualization image
    channel_axis=None             # Input is grayscale
)
print(f"HOG Feature Vector Dimension: {hog_features.shape[0]}")

# Step 6: Visualize HOG Representation
hog_image_rescaled = exposure.rescale_intensity(hog_image, in_range=(0, 10))
fig, (ax1, ax2) = plt.subplots(1, 2, figsize=(10, 5), sharex=True, sharey=True)
ax1.imshow(gray1, cmap=plt.cm.gray)
```

```
ax1.set_title('Original Grayscale Image')
ax1.axis('off')
ax2.imshow(hog_image_rescaled, cmap=plt.cm.gray)
ax2.set_title('HOG Gradient Orientation Visualization')
ax2.axis('off')
plt.show()
```

```
HOG Feature Vector Dimension: 190512
```

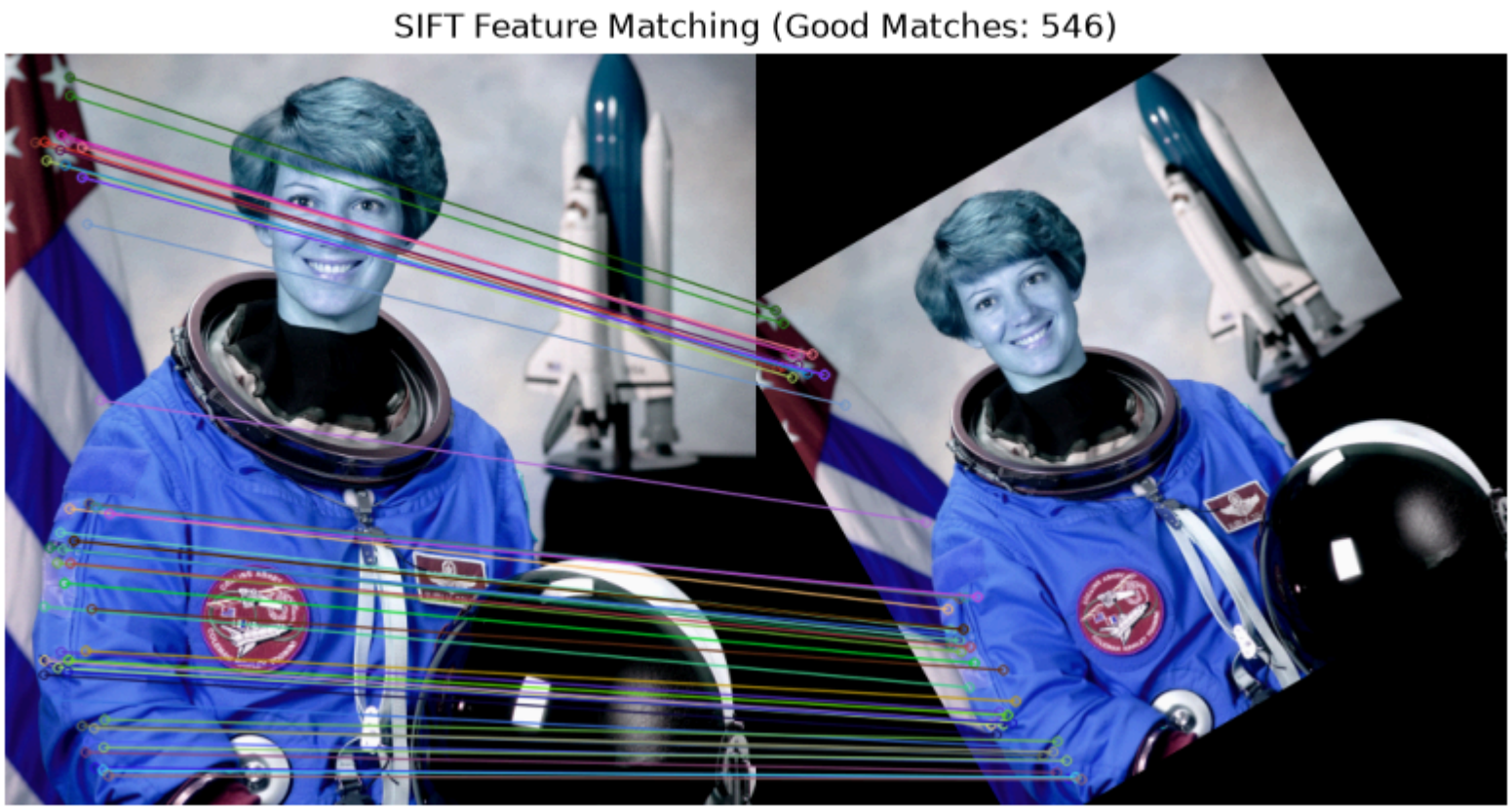


In [16]:

```
# Step 8: Image Matching using SIFT Descriptors and FLANN Matcher
FLANN_INDEX_KDTREE = 1
index_params = dict(algorithm=FLANN_INDEX_KDTREE, trees=5)
search_params = dict(checks=50)
flann = cv2.FlannBasedMatcher(index_params, search_params)
matches = flann.knnMatch(des1, des2, k=2)

good_matches = []
for m, n in matches:
    if m.distance < 0.7 * n.distance:
        good_matches.append(m)

img_matches = cv2.drawMatches(
    img1, kp1, img2, kp2, good_matches[:50], None,
    flags=cv2.DrawMatchesFlags_NOT_DRAW_SINGLE_POINTS
)
plt.figure(figsize=(10, 5))
plt.imshow(cv2.cvtColor(img_matches, cv2.COLOR_BGR2RGB))
plt.title(f"SIFT Feature Matching (Good Matches: {len(good_matches)})")
plt.axis('off')
plt.show()
```



Experiment 5: Analysis & Observations (Steps 7, 9, 10)

Step 7: Comparative Analysis of SIFT vs. HOG

Feature / Characteristic	SIFT (Scale-Invariant Feature Transform)	HOG (Histogram of Oriented Gradients)
Descriptor Type	Sparse local keypoint descriptor	Dense block-based gradient descriptor

Feature / Characteristic	SIFT (Scale-Invariant Feature Transform)	HOG (Histogram of Oriented Gradients)
Scale Invariance	**High** (Uses Difference-of-Gaussians scale-space)	**Low** (Requires manual image pyramids)
Rotation Invariance	**High** (Assigns dominant keypoint orientation)	**Low** (Sensitive to global rotation changes)
Illumination Invariance	**Moderate to High** (Normalized descriptor vectors)	**High** (Local block contrast normalization)
Computational Speed	Moderately slow (Keypoint detection + DoG filtering)	Fast / Structured (Ideal for real-time dense grids)
Representation	Local point-wise interest points (128-D vector)	Grid-based structural shape outlines

Step 9: Strengths & Limitations

1. SIFT

- **Strengths:** Robust to rotation, scaling, and affine transforms; ideal for sparse keypoint matching.
- **Limitations:** Higher computational cost; struggles on textureless surfaces.

2. HOG

- **Strengths:** Captures object structure/contours well; light-invariant; excellent for SVM classifiers.
- **Limitations:** Lacks intrinsic scale and rotation invariance; sensitive to fixed grid sizing.

Step 10: Suitable Practical Applications

1. **SIFT Applications:** Panorama stitching, 3D scene reconstruction, object matching under affine changes.
2. **HOG Applications:** Pedestrian detection, vehicle recognition, face detection/hand gesture analysis.